

Task1: To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

```
GNU nano 7.2 task1.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void handle_signal(int sig) {
    printf("Signal received: %d\n", sig);
}

int main() {
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);

    printf("Process PID: %d\n", getpid());
    printf("Process Group ID: %d\n", getpgrp());

    sigset_t mask;

    sigemptyset(&mask);
    sigaddset(&mask, SIGINT);

    sigprocmask(SIG_BLOCK, &mask, NULL);

    printf("SIGINT blocked for 10 seconds.\n");

    sleep(10);
```

```
sleep(10);

sigprocmask(SIG_UNBLOCK, &mask, NULL);

printf("SIGINT unblocked.\n");
printf("Send signals using another terminal.\n");

while (1) {
    sleep(1);
}

return 0;
```

```
shreyas@Grigori:~$ ps -o pid,ppid,pgid,sid,tpgid,state,cmd -p 1058
  PID   PPID   PGID   SID   TPGID S CMD
  1058   1030   1058   1030   1058 S ./task1
shreyas@Grigori:~$ kill -SIGINT PID
-bash: kill: PID: arguments must be process or job IDs
shreyas@Grigori:~$ kill -SIGINT 1058
shreyas@Grigori:~$ kill -SIGTERM 1058
shreyas@Grigori:~$ ./task1
Enter a command (type 'exit' to quit): ^C
shreyas@Grigori:~$ ps -o pid,ppid,pgid,sid,tpgid,state,cmd
  PID   PPID   PGID   SID   TPGID S CMD
  1066   1060   1066   1066   1085 S -bash
  1085   1066   1085   1066   1085 R ps -o pid,ppid,pgid,sid,tpgid,state,cmd
shreyas@Grigori:~$ kill -9 PID
-bash: kill: PID: arguments must be process or job IDs
shreyas@Grigori:~$ kill -9 1058
shreyas@Grigori:~$ |
```

```
shreyas@Grigori:~$ mkdir -p ~/signal_tasks
cd ~/signal_tasks
shreyas@Grigori:~/signal_tasks$ nano task1.c
shreyas@Grigori:~/signal_tasks$ gcc task1.c -o task1
shreyas@Grigori:~/signal_tasks$ ./task1
Process PID: 1058
Process Group ID: 1058
SIGINT blocked for 10 seconds.
SIGINT unblocked.
Send signals using another terminal.
Signal received: 2
Signal received: 15
Killed
shreyas@Grigori:~/signal_tasks$ |
```

Task2: To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery

```

        while (1) {
            printf("Job is running...\n");
            sleep(2);
        }
    }

    job.job_id = 1;
    job.pid = pid;
    job.state = 1;

    printf("Job created.\n");
    printf("Job ID: %d\n", job.job_id);
    printf("PID: %d\n", job.pid);

    sleep(3);

    printf("\nStopping job...\n");

    kill(job.pid, SIGSTOP);
    job.state = 0;

    sleep(2);

    printf("Job stopped.\n");

```

```

shreyas@Grigori:~/signal_tasks$ nano task2.c
shreyas@Grigori:~/signal_tasks$ gcc task2.c -o task2
shreyas@Grigori:~/signal_tasks$ ./task2
Job created.
Job ID: 1
PID: 1101
Background job started.
PID: 1101
Job is running...
Job is running...

Stopping job...
Job stopped.

Sending SIGCONT...
Job resumed.
Job is running...
Job is running...
Job is running...

Stopping background job...
Job state updated to STOPPED.
Recovering background job...
Job resumed successfully.
Job is running...
Job is running...
Job is running...
Terminating job...
Job completed and removed from job table.
shreyas@Grigori:~/signal_tasks$ sleep 60 &

```

```

shreyas@Grigori:~$ ps -o pid,ppid,pgid,state,cmd
  PID   PPID   PGID S CMD
  1108   1103   1108 S -bash
  1124   1108   1124 R ps -o pid,ppid,pgid,state,cmd
shreyas@Grigori:~$ sleep 60 &
[1] 1125
shreyas@Grigori:~$ jobs -l
[1]+  1125 Running                  sleep 60 &
shreyas@Grigori:~$ kill -STOP 1125

[1]+  Stopped                      sleep 60
shreyas@Grigori:~$ sleep 30 &
[2] 1127
shreyas@Grigori:~$ kill -STOP 1125
shreyas@Grigori:~$ kill -CONT 1125
shreyas@Grigori:~$ jobs
[1]-  Done                          sleep 60
[2]+  Running                      sleep 30 &
shreyas@Grigori:~$ kill 1125
-bash: kill: (1125) - No such process
[2]+  Done                          sleep 30
shreyas@Grigori:~$ |

```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

typedef struct {
    int job_id;
    pid_t pid;
    int state;
} Job;

int main() {
    Job job;

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Background job started.\n");
        printf("PID: %d\n", getpid());
    }
}

```